

# Reviewer Pack — Minimal Rank of Formal Language Automorphism Groups Bounds A...

Entry 4dc1555531ec · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 02:00:26 UTC

## Minimal Rank of Formal Language Automorphism Groups Bounds AC0 Parity Depth

Entry ID: 4dc1555531ec **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 02:00:26 UTC

### 1. Conjecture

**Field A** (mathematical branch): Formal Language Theory (Automorphism Groups) **Field B** (complexity object): Complexity Theory: Boolean Circuit Monotonicity

#### Statement:

[‘For any language  $L$  in the class of regular languages, the minimal rank of the automorphism group of  $L$ , denoted as  $\text{rank\_autom}(L)$ , satisfies  $\text{rank\_autom}(L) \geq c \cdot \log(\text{size}(A0\_n))$ , where  $A0\_n$  is an AC0 circuit computing PARITY on  $n$  inputs and  $\text{size}(A0\_n)$  is its size.’, ‘For any language  $L$  not in the class of regular languages,  $\text{rank\_autom}(L) \geq d \cdot n^\alpha$  for some constant  $d > 1$  and  $\alpha < 1/3$ .’, ‘These bounds are tight: there exist regular languages with  $\text{rank\_autom}(L) = c \cdot \log(\text{size}(A0\_n))$  and non-regular languages with  $\text{rank\_autom}(L) = d \cdot n^\alpha$ .’]

#### Rationale (proposer’s reasoning):

[‘Automorphism groups capture the symmetries of a language, which might reflect intrinsic complexity properties. A higher rank could indicate more complex interactions within the language that affect its computational properties.’, ‘The proposed invariant could reveal hidden structures in AC0 circuits computing PARITY, providing a new perspective on monotonicity lower bounds that goes beyond simple structural descriptions or random restrictions.’]

**Taxonomy category:** AC0\_PARITY (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 6b16f70d351f77ed

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

For regular languages, the correlation between  $\text{rank\_autom}(L)$  and  $\text{size}(A0\_n)$  must be  $\geq 0.8$  AND  $\text{metric\_mean} \leq 3$  for all seeds; for non-regular languages, the correlation must be  $\geq 0.8$  AND no seed has  $\text{rank\_autom}(L) < d \cdot n^\alpha$ .

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | UNCERTAIN        | HITS             |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_dfa(n):
        states = list(range(2**n))
        transitions = {state: {} for state in states}
        accepting_states = [0]
        for state in states:
            for bit in range(n):
                next_state = (state >> 1) | ((bit ^ (state & 1)) << (n - 1))
                transitions[state][bit] = next_state
        return states, transitions, accepting_states

    def generate_cnf(n):
        variables = list(range(1, n + 1))
        clauses = []
        for i in range(2**n):
            clause = []
            for j in range(n):
                if (i >> j) & 1:
                    clause.append(variables[j])
                else:
                    clause.append(-variables[j])
            clauses.append(clause)
        return variables, clauses

    def ac0_circuit_size(n):
        # Simplified approximation of AC0 circuit size for PARITY
        return n * math.log2(n)
```

```

def automorphism_group(dfa):
    states = dfa[0]
    transitions = dfa[1]
    group = []
    for perm in itertools.permutations(states):
        is_automorphism = True
        for state in states:
            for bit in range(len(state)):
                if transitions[state][bit] != perm[transitions[perm[state]][bit]]:
                    is_automorphism = False
                    break
            if not is_automorphism:
                break
        if is_automorphism:
            group.append(perm)
    return group

def rank_of_group(group):
    return len(group)

n_values = [5, 10, 15, 20, 30, 40]
metric_value = 0.0
instances_tested = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    # Generate random regular language using DFA
    dfa = generate_dfa(n)
    rank_regular = rank_of_group(automorphism_group(dfa))
    size_ac0 = ac0_circuit_size(n)
    metric_value += abs(rank_regular - c * math.log(size_ac0))

    # Generate non-regular language using CNF
    cnf = generate_cnf(n)
    rank_non_regular = rank_of_group(automorphism_group(cnf))
    if rank_non_regular < d * n**alpha:
        counterexample = f"Non-regular language with rank {rank_non_regular} and n={n}"
        conjecture_holds = False
    metric_value += abs(rank_non_regular - d * n**alpha)

    instances_tested += 2

return {
    "metric_name": "Rank of Automorphism Group",
    "metric_value": metric_value / instances_tested,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)] # Default to first

    results = []

```

```

for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")
    results.append(trial_result)

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)]['counterexample']}\")")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_312341fe.py", line 105, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_312341fe.py", line 77, in run_trial
    rank_regular = rank_of_group(automorphism_group(dfa))
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_312341fe.py", line 55, in automorphism_group
    for bit in range(len(state)):
                   ^^^^^^^^^^^
TypeError: object of type 'int' has no len()

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which means that the pre-registered support conditions could not be verified. | next: Investigate and fix the error in the test code to ensure it can run to completion and verify the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 10

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 11256        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 11467        |
| 3  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 6327         |
| 4  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4796         |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 5202         |
| 6  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 22583        |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15858        |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17966        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11031        |
| 10 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 8105         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 114590 ms total latency. Provider mix: {'ollama\_remote': 10}

(full prompt+response transcripts available in *research/audit/4dc155531ec.jsonl*)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/4dc155531ec.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** *research/pvsnp\_sandbox/test\_\*.py* (per-cycle)

**Replay tarball:** *research/replay/4dc155531ec.tar.gz* (if generated)